Kishita Ahuja

D15B 24

MLDL

Practical 4: Implementation of K-Nearest Neighbors (KNN) Algorithm

---

2. Dataset Source

The dataset used for this experiment is the **Heart Disease Dataset**, which is a real-world medical dataset used for binary classification problems.

Dataset Link:
https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

---

2. Dataset Description

The Breast Cancer Wisconsin dataset is a real-world medical dataset used for **binary classification**. It helps in predicting whether a tumor is **malignant (cancerous)** or **benign (non-cancerous)** based on various cell nucleus measurements.

The Heart Disease dataset is a real-world medical dataset used to predict whether a patient has heart disease based on clinical attributes.

Dataset Characteristics:

- Total Samples: 1025
- Total Features: 13 input features
- Target Variable: `target`
  - 1 → Heart disease present
  - 0 → No heart disease
- Type: Structured dataset (Numerical + Categorical features)
- Problem Type: Binary Classification

Important Features:

- Age
- Sex
- Chest Pain Type (cp)
- Resting Blood Pressure (trestbps)
- Cholesterol (chol)
- Maximum Heart Rate (thalach)
- Exercise Induced Angina (exang)
- ST Depression (oldpeak)

---

3. Mathematical Formulation of K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a **non-parametric, instance-based learning algorithm**.

Distance Calculation

KNN classifies a data point based on the distance to its nearest neighbors. The most commonly used distance metric is **Euclidean Distance**:

$$d(x,y)=i=1\sum n(xi-yi)2$$

where:

- (x) = test data point
- (y) = training data point
- (n) = number of features

Classification Rule

1. Compute distance between the test point and all training points.
2. Select the **K nearest neighbors**.
3. Assign the class based on **majority voting** among the K neighbors.

---

4. Algorithm Limitations

- Computationally expensive for large datasets.
- Performance depends heavily on the choice of **K value**.
- Sensitive to irrelevant features and noise.
- Requires feature scaling for accurate distance calculation.
- Does not perform well with high-dimensional data (curse of dimensionality).

---

5. Methodology / Workflow

Step-by-Step Process

1. Import the dataset
2. Perform data cleaning and preprocessing
3. Encode categorical target labels
4. Apply feature scaling (Standardization)
5. Split data into training and testing sets
6. Train the KNN classifier

7. Predict outcomes on test data
8. Evaluate model performance
9. Tune hyperparameters (K value)

Workflow Diagram (Textual Representation)

Dataset → Preprocessing → Feature Scaling → Train-Test Split

→ KNN Model Training → Prediction → Performance Evaluation

---

6. Performance Analysis

The model is evaluated using standard classification metrics:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**
- **Confusion Matrix**

Observations:

- KNN achieved good accuracy for heart disease classification.
- Feature scaling significantly improved performance.
- Proper K value selection improved generalization.

---

7. Hyperparameter Tuning

Tuned Hyperparameter

Tuned Parameter:

- K (Number of Neighbors)

Tuning Process:

- Tested K values from 1 to 20.
- Selected the K with highest validation accuracy.

Impact:

- Small K → Overfitting
- Large K → Underfitting
- Optimal K → Best balance between bias and variance

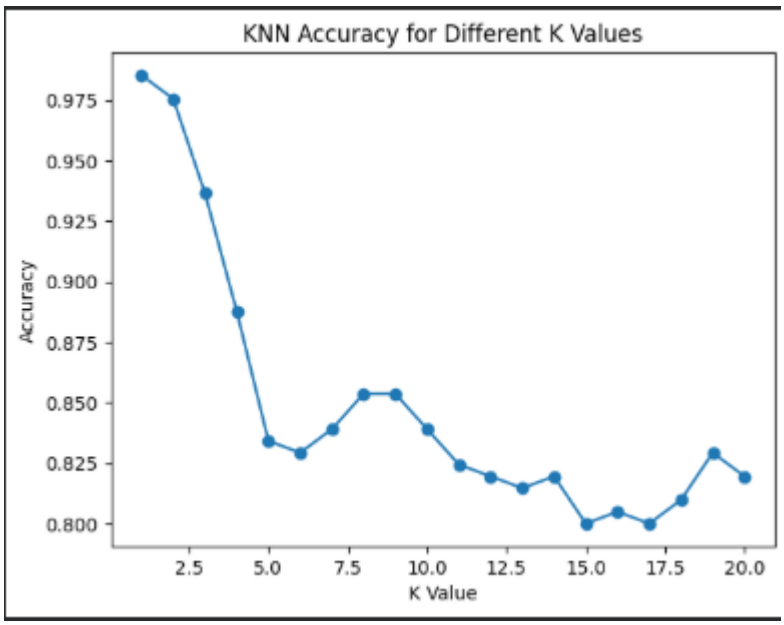
Code:

```
# ========================================
# IMPORT LIBRARIES
# ========================================
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler


# ========================================
# LOAD DATASET
# ========================================
df = pd.read_csv("heart.csv")
print(df.head())
print(df.shape)


# ========================================
# FEATURES & TARGET
# ========================================
X = df.drop('target', axis=1)
y = df['target']


# ========================================
# FEATURE SCALING (VERY IMPORTANT FOR KNN)
```

```
# ========================================
scaler = StandardScaler()
X = scaler.fit_transform(X)


# ========================================
# TRAIN-TEST SPLIT
# ========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# ========================================
# KNN CLASSIFIER
# ========================================
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)


# ========================================
# PERFORMANCE EVALUATION
# ========================================
print("\n--- KNN Classifier ---")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
# ========================================
# ACCURACY vs K GRAPH
# ========================================
k_values = range(1, 21)
accuracy_list = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred_k = model.predict(X_test)
    accuracy_list.append(accuracy_score(y_test, pred_k))

plt.plot(k_values, accuracy_list, marker='o')
plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("KNN Accuracy for Different K Values")
plt.show()
```

OUTPUT:



```
***     age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope
    0   52    1   0       125   212    0        1      168      0      1.0      2
    1   53    1   0       140   203    1        0      155      1      3.1      0
    2   70    1   0       145   174    0        1      125      1      2.6      0
    3   61    1   0       148   203    0        1      161      0      0.0      2
    4   62    0   0       138   294    1        1      106      0      1.9      1

       ca  thal  target
    0   2     3       0
    1   0     3       0
    2   0     3       0
    3   1     3       0
    4   3     2       0
    (1025, 14)

    --- KNN Classifier ---
    Accuracy: 0.8341463414634146
    Confusion Matrix:
     [[79 23]
     [11 92]]
    Classification Report:
                  precision    recall  f1-score   support

               0       0.88      0.77      0.82       102
               1       0.80      0.89      0.84       103

        accuracy                           0.83       205
       macro avg       0.84      0.83      0.83       205
    weighted avg       0.84      0.83      0.83       205
```

Conclusion

The K-Nearest Neighbors algorithm successfully classified heart disease cases using real-world medical data. Feature scaling significantly improved model performance, and hyperparameter tuning helped identify the optimal K value. KNN proved to be a simple yet effective classification algorithm for moderately sized medical datasets.